

QUESTIONS 10 / 10 completed

Stack using Array ▾

STACK USING ARRAY

"A web browser maintains a history of recently visited pages. Write a C program to implement a stack using an array and perform Push, Pop, Peek, and Display operations while handling overflow and underflow conditions. Test Case 1

PROGRAM EDITOR

C ▾

Run

Save

```
1 #include <stdio.h>
2 #define MAX 5
3
4 int stack[MAX], top = -1;
5
6 void push(int value)
7 {
8     if (top == MAX - 1)
9         printf("Stack Overflow\n");
10    else
11    {
12        stack[++top] = value;
13        printf("%d pushed\n", value);
14    }
15 }
16
17 void pop()
18 {
19     if (top == -1)
20         printf("Stack Underflow\n");
21     else
```

INPUT

5
10 20 30

OUTPUT

```
10 pushed
20 pushed
30 pushed
Stack: 30 20 10
Top element: 30
30 popped
Stack: 20 10
```

QUESTIONS 10 / 10 completed

Stack using Linked

STACK USING LINKED LIST

"A document editor maintains recently performed actions to support the Undo feature. Implement a stack using a linked list and perform push, pop, and display operations.

Test Case 1

Input

PROGRAM EDITOR

C

Run

Save

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 struct Node {
5     int data;
6     struct Node *next;
7 };
8
9 struct Node *top = NULL;
10
11 void push(int value)
12 {
13     struct Node *newNode = malloc(sizeof(struct Node));
14
15     newNode->data = value;
16     newNode->next = top;
17     top = newNode;
18 }
19
20 void pop()
21 {
```

INPUT

Your INPUT goes here!

OUTPUT

```
Stack: 300 200 100
```

Stack -Postfix Expr

STACK -POSTFIX EXPRESSION

"Write a C program that uses a stack to evaluate a postfix expression and display the result.

Postfix expressions are commonly used in compilers because they eliminate the need for parentheses.

Input

23*5+

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 int stack[20], top = -1;
5
6 void push(int x)
7 {
8     stack[++top] = x;
9 }
10
11 int pop()
12 {
13     return stack[top--];
14 }
15
16 int main()
17 {
18     char exp[20];
19     int i, a, b;
20
21     scanf("%s", exp);
```

23*5+

OUTPUT

Security Error: Disallowed code pattern detected.

Stack- Undo Opera ▾

STACK- UNDO OPERATION

"A text editor provides an Undo feature that allows users to revert the most recent action. Write a C program using a stack to simulate an Undo operation for text editing commands
Input

Insert A
Insert B ▾

```
1 #include <stdio.h>
2
3 char stack[100];
4 int top = -1;
5
6 void push(char c)
7 {
8     stack[++top] = c;
9 }
10
11 char pop()
12 {
13     return stack[top--];
14 }
15
16 int main()
17 {
18     char command;
19
20     push('A');
21     push('B');
```

OUTPUT

```
Undo: Insert C
Remaining Text: AB
```

Your INPUT goes here!

OUTPUT

```
Move Disk 1 from A to C
Move Disk 2 from A to B
Move Disk 1 from C to B
Move Disk 3 from A to C
Move Disk 1 from B to A
Move Disk 2 from B to C
Move Disk 1 from A to C
```


Stack - Browser His ▾

STACK - BROWSER HISTORY

“A web browser stores visited web pages in a stack to enable navigation to previously visited pages. Write a C program that simulates browser history using a stack and supports back navigation.

Input

```
1 #include <stdio.h>
2 #include <string.h>
3
4 char stack[10][20];
5 int top = -1;
6
7 void visit(char page[])
8 {
9     top++;
10    strcpy(stack[top], page);
11 }
12
13 void back()
14 {
15     if (top > 0)
16     {
17         top--;
18         printf("Back to: %s\n", stack[top]);
19     }
20     else
21     {
```

OUTPUT

```
Current Page: Wikipedia
Back to: YouTube
```

Your INPUT goes
here!

Tower of Hanoi Prc ▾

TOWER OF HANOI PROBLEM

"The Tower of Hanoi is a classic problem demonstrating recursion and stack behavior. Write a C program to solve the Tower of Hanoi problem and display the sequence of moves required to transfer all disks. Input

```
1 #include <stdio.h>
2
3 void hanoi(int n, char from, char to, char aux)
4 {
5     if (n == 1)
6     {
7         printf("Move Disk 1 from %c to %c\n", from, to);
8         return;
9     }
10
11     hanoi(n - 1, from, aux, to);
12     printf("Move Disk %d from %c to %c\n", n, from, to);
13     hanoi(n - 1, aux, to, from);
14 }
15
16 int main()
17 {
18     int n;
19
20     scanf("%d", &n);
21
```

3

Stack -Infix to Posti ▾

STACK -INFIX TO POSTFIX

"Arithmetic expressions entered by programmers are generally in infix form. Before evaluation, many systems convert them into postfix notation. Write a C program using a stack to convert an infix expression into its equivalent postfix expression."

```
1 #include <stdio.h>
2 #include <ctype.h>
3
4 char stack[100];
5 int top = -1;
6
7 void push(char c)
8 {
9     stack[++top] = c;
10 }
11
12 char pop()
13 {
14     return stack[top--];
15 }
16
17 int priority(char c)
18 {
19     if (c == '+' || c == '-')
20         return 1;
21     if (c == '*' || c == '/')
```

A+B*C

OUTPUT

Security Error: Disallowed code pattern detected.

Stack- Balanced Pa ▾

STACK-BALANCED PARENTHESES

"Compilers must verify whether all opening parentheses have corresponding closing parentheses. Write a C program using a stack to check whether an expression contains balanced parentheses. Input

(A+B)*(C-D)

```
1 #include <stdio.h>
2
3 char stack[100];
4 int top = -1;
5
6 void push(char c)
7 {
8     stack[++top] = c;
9 }
10
11 char pop()
12 {
13     return stack[top--];
14 }
15
16 int main()
17 {
18     char exp[100];
19     int i, balanced = 1;
20
21     scanf("%s", exp);
```

(A+B)*(C-D)

OUTPUT

Security Error: Disallowed code pattern detected.

Stack - Reverse Stri ▾

STACK - REVERSE STRING

"A text-processing application requires reversing user-entered strings. Write a C program that uses a stack to reverse a given string.

Input

DATASTRUCTURE

Output

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main()
5 {
6     char str[100], stack[100];
7     int top = -1, i;
8
9     scanf("%s", str);
10
11     for (i = 0; str[i] != '\0'; i++)
12         stack[++top] = str[i];
13
14     while (top >= 0)
15         printf("%c", stack[top--]);
16
17     return 0;
18 }
```

Data structure

OUTPUT

Security Error: Disallowed code pattern detected.

STACK- DECIMAL TO BINARY

"Computer systems internally represent numbers in binary format.. Write a C program that uses a stack to convert a decimal number into its binary equivalent.
Input

13

Output

```
1 #include <stdio.h>
2
3 int main()
4 {
5     int n, stack[20], top = -1;
6
7     scanf("%d", &n);
8
9     while (n > 0)
10    {
11        stack[++top] = n % 2;
12        n = n / 2;
13    }
14
15    while (top >= 0)
16    {
17        printf("%d", stack[top--]);
18    }
19
20    return 0;
21 }
```

OUTPUT

1101